# Aira: Technical Setup Guide

Nov 10, 2026

## Overview

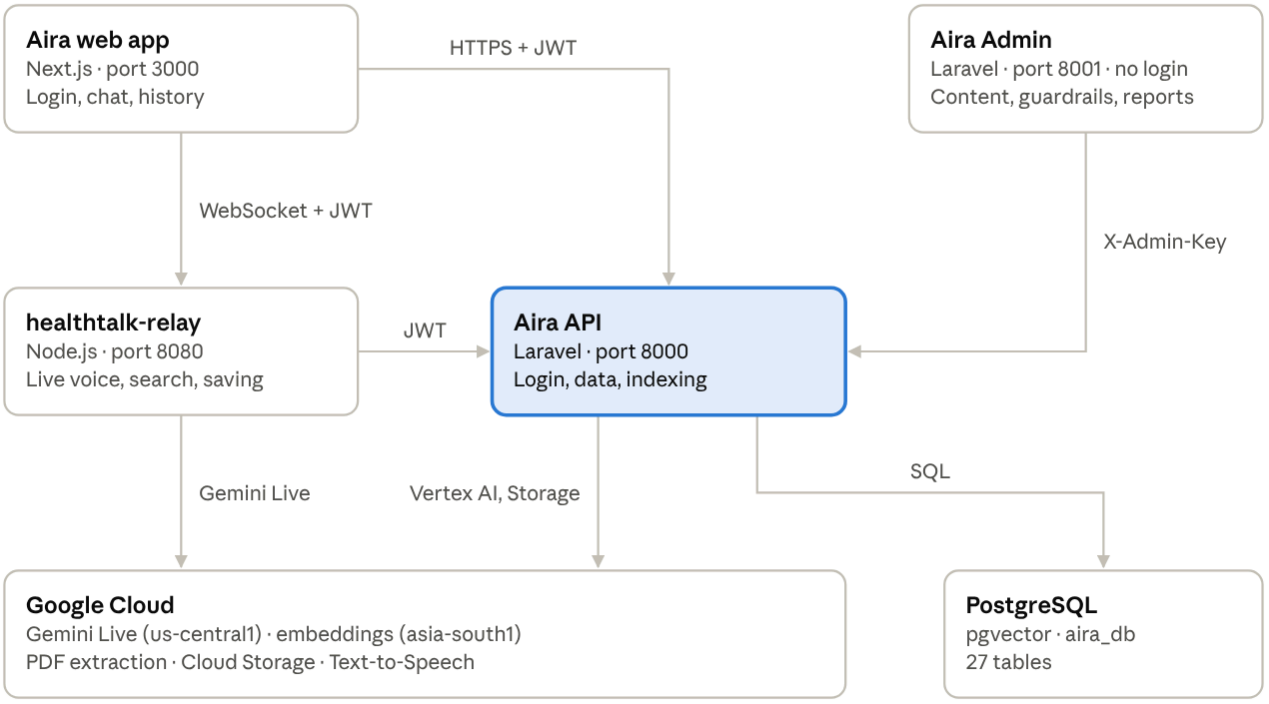
Aira is Saathealth's voice-first AI health assistant. Users talk or type to it in English, Hindi, Marathi or Bengali, and it answers from a Content Library that the Saathealth team manages. It runs as four separate projects:

- **Aira web app** (Next.js): what users see. Mobile number and OTP login, registration, profile, the voice and text chat, and past conversations.
- **healthtalk-relay** (Node.js): a WebSocket server between the browser and Google's Gemini Live model. It streams the user's voice in and Aira's voice out, adds the guardrails, searches the Content Library and saves each conversation. It keeps the Google service account off the browser.
- **Aira API** (Laravel, `saathi-laravel-api`): owns the database. It handles login, profiles, saved conversations, Content Library indexing and search, guardrails, the introductory message and reports.
- **Aira Admin** (Laravel, `saathi-laravel-admin`): the internal panel where the team edits the Content Library, guardrails and introductory message, reads conversation reports and lists users. It has no database tables of its own and reads everything through the API.

The Aira web app and the relay call the API with the user's login token. The admin panel calls the API server to server with a shared admin key. The API stores everything in one PostgreSQL database with the pgvector extension.

These standalone copies keep only the Aira features. The other Saathi modules, the old admin login, the dashboard and the database migrations have been removed.

**Only the API touches the database; the relay keeps Google keys off the browser**




Aira architecture · 4 projects, 2 shared services

The web app and the relay act on the user's behalf with the user's token; the admin panel uses its own key; only the API reads and writes PostgreSQL.

## Repository layout

All four projects live in one GitHub repository, one project per branch. The branches share no history, so check out the branch you need, or clone it on its own:

```
git clone -b <branch> --single-branch
https://github.com/AditiHazraGanju/AIRA.git <folder>
```

| Branch | Project | Stack | Local port | Run command |
|---|---|---|---|---|
| `aira-frontend` | Aira web app | Next.js 16, React 19, Node.js 20+ | 3000 | `npm run dev` |
| `aira-relay` | WebSocket relay to Gemini Live | Node.js 20+ | 8080 | `npm start` |

| Branch | Project | Stack | Local port | Run command |
|---|---|---|---|---|
| `aira-api` | Aira API | Laravel 12, PHP 8.4, PostgreSQL + pgvector | 8000 | `php artisan serve --port=8000` |
| `aira-admin` | Aira Admin panel | Laravel 12, PHP 8.4, Vite | 8001 | `php artisan serve --port=8001` |

The repository is github.com/AditiHazraGanju/AIRA. Each branch has its own `README.md` and a `.env.example` that documents every setting. No branch contains real keys or passwords: every secret is a `YOUR_*` placeholder.

## Prerequisites

Install the software below and collect the accounts and keys before you start; the API and the relay don't work without the Google Cloud service account.

### Software

| Tool | Version | Needed for |
|---|---|---|
| PHP | 8.4, with `pdo_pgsql`, `openssl`, `mbstring`, `curl`, `fileinfo` | API, Admin |
| Composer | 2.x | API, Admin |
| Node.js and npm | 20 or later | Aira web app, relay, Admin (asset build) |
| PostgreSQL | 15 or later | API (Admin shares the same database) |
| pgvector extension | Installed on the PostgreSQL server | API (Content Library search) |
| Git | Any recent | All |

### Accounts and keys

| What | Used by | Where to get it |
|---|---|---|
| Google Cloud project with Vertex AI, Cloud Storage and Text-to-Speech enabled | API, relay | Google Cloud Console |
| Service-account JSON key with Vertex AI User, Storage Object Admin on the bucket and Text-to-Speech access | API, relay | IAM & Admin > Service Accounts > Keys > Add key > JSON |
| A Cloud Storage bucket | API (uploaded PDFs, introductory-message audio) | Cloud Storage > Create bucket |
| SMS Gateway Hub account (SMS and WhatsApp OTP) | API | smsgatewayhub.com dashboard. Not needed for local development. |
| TCM logger credentials | API, Admin, Aira web app | Optional. Leave remote logging off without them. |

### Secrets you generate yourself

| Value | Generate with | Must match in |
|---|---|---|
| `JWT_SECRET` | `php artisan jwt:secret` in the API | API and relay |
| `ADMIN_API_KEY` | `openssl rand -hex 32` | API and Admin |
| `APP_KEY` (one each) | `php artisan key:generate` | Not shared. The API's must stay the same once users exist. |

## Aira API (branch `aira-api`)

The API owns the database and every piece of business logic; set it up first, because the other three projects depend on it.

## What it does

- **Login:** sends a 6-digit OTP by SMS and WhatsApp, verifies it, registers new users and issues a JWT. Registration only works within 30 minutes of a verified OTP for that number, and an OTP is voided after 5 wrong tries.
- **Profile:** reads and updates the user's details. For users who already exist in the Saathi beneficiary records, it fills the profile from them on login.
- **Conversations:** stores the sessions and messages that the relay sends, and records ratings and answer feedback. When a session ends, it uses Gemini to write the title, summary, tags and Q&A pairs.
- **Content Library:** stores text and PDF content. After each save it extracts the text (Gemini reads the PDF from Cloud Storage), splits it into chunks, embeds them with `gemini-embedding-001` and stores the vectors in PostgreSQL. It also answers the relay's vector searches.
- **Guardrails and introductory message:** stores both; for the introductory message it also generates audio with Google Text-to-Speech.
- **Reports and SH Users:** serves the admin panel's conversation statistics and user list.

It answers three kinds of caller:

| Caller | Authentication | Endpoints |
|---|---|---|
| Anyone | None | `POST /api/v1/auth/send-otp`, `POST /api/v1/auth/chatbot/verify-otp`, `POST /api/v1/auth/chatbot/register`, `GET /api/v1/auth/states`, `GET /up` |
| Aira web app and relay | `Authorization: Bearer <JWT>` | Profile, sessions, ratings, Content Library search, guardrails and introductory message (read) |
| Admin panel | `X-Admin-Key: <ADMIN_API_KEY>` | Content Library, guardrails, introductory message (edit), reports, SH Users, plus the guardrail and introductory-message reads |

`php artisan route:list` prints all 35 routes.

## Setup

1. Check out the branch and install the PHP packages.

```
git clone -b aira-api --single-branch
https://github.com/AditiHazraGanju/AIRA.git aira-api
cd aira-api
composer install
```

2. Create the environment file and fill in every `YOUR_*` value (see the variables below).

```
cp .env.example .env
php artisan key:generate    # skip if you are reusing an existing APP_KEY
php artisan jwt:secret      # writes JWT_SECRET into .env
```

3. Create the database. There are no migrations: one SQL file creates all 27 tables, their indexes and foreign keys, and the two default settings rows. Then seed the 36 states for the registration form (safe to run again).

```
createdb aira_db
psql -d aira_db -v ON_ERROR_STOP=1 -f database/sql/aira_schema.sql
php artisan db:seed
```

   To use an existing Aira database instead, skip this step and point the `DB_*` settings at it.

4. Start the server.

```
php artisan serve --port=8000
```

5. Check it: `curl http://localhost:8000/up` returns 200. `curl -H "Accept: application/json" http://localhost:8000/api/v1/healthtalk/content/all` returns 401, because there is no admin key.

Content Library indexing runs right after the HTTP response, in the same PHP process, so no queue worker is needed. To index by hand, run `php artisan healthtalk:index-content` for pending and failed items, or add `--all` to re-index everything.

## Environment variables

| Variable | What to put | Example |
|---|---|---|
| `APP_ENV` | `local`, `staging` or `production`. With `local`, the OTP is always 102025 and nothing is sent. | `production` |
| `APP_KEY` | Laravel key. It also encrypts users' mobile numbers, so keep it once users exist. | `base64:…` |

| Variable | What to put | Example |
|---|---|---|
| `APP_URL` | Public URL of the API, no trailing slash | `https://api.example.com` |
| `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD` | The PostgreSQL database | `aira_db` |
| `JWT_SECRET` | Signs user tokens. The relay needs the same value. | from `php artisan jwt:secret` |
| `JWT_TTL` | Token lifetime in minutes | `525600` (1 year) |
| `ADMIN_API_KEY` | Shared secret with the admin panel | from `openssl rand -hex 32` |
| `SMSGATEWAYHUB_SMS_API_KEY`, `SMSGATEWAYHUB_SENDER_ID`, `SMSGATEWAYHUB_MESSAGE_URL`, `SMSGATEWAYHUB_DEV_API_KEY`, `SMSGATEWAYHUB_WHATSAPP_OTP_TEMPLATE` | OTP delivery by SMS and WhatsApp | from the SMS Gateway Hub dashboard |
| `GOOGLE_CLOUD_PROJECT_ID` | Google Cloud project ID | `my-project-123456` |
| `GOOGLE_CLOUD_KEY_FILE`, `GOOGLE_APPLICATION_CREDENTIALS` | Absolute path to the service-account JSON key, outside `public/` | `/etc/secrets/gcp.json` |
| `GOOGLE_CLOUD_STORAGE_BUCKET` | Bucket name, without `gs://` | `aira-content` |
| `VERTEX_LOCATION`, `VERTEX_EMBEDDING_MODEL`, `VERTEX_EXTRACTION_MODEL` | Vertex AI region and models. The embedding model must match the relay's. | `asia-south1`, `gemini-embedding-001`, `gemini-2.5-flash` |
| `TCMLOGGER_*` | Optional remote logging. `TCMLOGGER_DEBUG=false` turns it off. | |

## Aira Admin (branch `aira-admin`)

The admin panel is where the Saathealth team controls what Aira says. It has no login, so it must never be reachable from the public internet.

## What it does

| Page | URL | Purpose |
|---|---|---|
| Content Library | `/healthtalk/content` | Add, edit, enable or delete the text and PDFs Aira may answer from. Each item shows whether it is Ready, Indexing or Failed. |
| Guardrails | `/healthtalk/guardrails` | Topics Aira must refuse, plus the message she gives when she refuses, per language |
| Introductory Message | `/healthtalk/start-pitch` | Aira's opening message per language, with generated audio and a preview |
| Conversation Reports | `/healthtalk/reports` | Usage, ratings, answers users marked wrong, and questions the library couldn't answer, with full transcripts |
| SH Users | `/sh-users` | Everyone in the central users table |

`/` redirects to the Content Library. Every page reads and writes through the API, server to server, sending `ADMIN_API_KEY` in the `X-Admin-Key` header. The browser never sees the key. The panel uses the database only for its own sessions, cache and jobs tables.

## Setup

1. Check out the branch and install the PHP and JavaScript packages.

```
git clone -b aira-admin --single-branch
https://github.com/AditiHazraGanju/AIRA.git aira-admin
cd aira-admin
composer install
npm install
npm run build      # builds public/build (CSS, JS and UI plugins)
```

2. Create the environment file and fill in every `YOUR_*` value.

```
cp .env.example .env
php artisan key:generate
```

3. Make sure the API is running, then start the panel.

```
php artisan serve --port=8001
```

4. Open `http://localhost:8001`. The Content Library page should load. If every page shows empty tables, the admin key doesn't match the API's (see Troubleshooting).

There is nothing to migrate or seed: the tables come from the API's `aira_schema.sql`.

### Environment variables

| Variable | What to put | Example |
|---|---|---|
| `APP_KEY` | Laravel key for cookies and CSRF. Not shared with the API. | `base64:…` |
| `APP_URL` | Public URL of the panel | `https://admin.example.com` |
| `AIRA_API_URL` | API base URL, without `/api/v1` and without a trailing slash | `http://127.0.0.1:8000` |
| `ADMIN_API_KEY` | Exactly the same value as the API's `ADMIN_API_KEY` | from `openssl rand -hex 32` |
| `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD` | The same database as the API, for sessions, cache and jobs | `aira_db` |
| `APP_VERSION` | Version shown in the footer | `1.0.0` |
| `TCMLOGGER_*` | Optional remote logging. `TCMLOGGER_DEBUG=false` turns it off. | |

## Relay server (branch `aira-relay`)

The relay carries every live conversation between the browser and Gemini Live, so it must run as a long-lived Node.js process that can hold WebSocket connections open.

### What it does

- **Checks the user:** the first message on the socket must carry the user's JWT and language (`ENG`, `HIN`, `MAR` or `BEN`). The relay verifies the token with `JWT_SECRET` and closes the socket with code 4003 if it is invalid.
- **Builds Aira's instructions:** a language lock plus the guardrails and their refusal message, fetched from the API and cached for `GUARDRAIL_CACHE_SECONDS`.

- **Streams audio:** microphone audio in (PCM16, 16 kHz) and Aira's voice out (PCM16, 24 kHz), with live transcripts. When the user talks over Aira, it tells the browser to stop playback.
- **Searches the Content Library:** when Gemini asks for a lookup, the relay embeds the question with `gemini-embedding-001` and sends the vector to the API's search endpoint with the user's token. It returns up to 4 passages to Gemini.
- **Saves the conversation:** after each turn it posts the messages to the API, and at the end it asks the API to summarise the session.
- **Limits:** at most `MAX_CONCURRENT_SESSIONS` conversations, each at most `MAX_SESSION_MINUTES`, closed after `IDLE_TIMEOUT_SECONDS` without input.

Gemini Live runs in `us-central1`, because no Live model is served from `asia-south1`. Embeddings run in `asia-south1`.

## Setup

1. Check out the branch and install the packages.

```
git clone -b aira-relay --single-branch
https://github.com/AditiHazraGanju/AIRA.git aira-relay
cd aira-relay
npm install
```

2. Create the environment file and fill in every `YOUR_*` value.

```
cp .env.example .env
```

3. Start the relay. Use `npm run dev` instead to restart on file changes.

```
npm start      # ws://localhost:8080/ws, health check at
http://localhost:8080/healthz
```

4. Check it: `http://localhost:8080/healthz` responds, and the startup log line `relay listening` shows a `jwtSecretFingerprint`. With the API running, `npm run test:client` runs a full test conversation (spoken turn, typed turn, interruption and guardrail).

## Environment variables

| Variable | What to put | Example |
|---|---|---|
| `PORT` | Port for the WebSocket (`/ws`) and health check (`/healthz`) | `8080` |

| Variable | What to put | Example |
|---|---|---|
| `ALLOWED_ORIGINS` | Comma-separated URLs the Aira web app is served from. Empty allows any origin, so use that for local development only. | `https://aira.example.com` |
| `JWT_SECRET` | Exactly the API's `JWT_SECRET` | |
| `LARAVEL_API_URL` | API base URL including `/api/v1` | `http://localhost:8000/api/v1` |
| `GOOGLE_APPLICATION_CREDENTIALS` | Absolute path to the service-account JSON key (role Vertex AI User). Leave empty on Cloud Run or GCE with an attached service account. | `/etc/secrets/gcp.json` |
| `GCP_PROJECT_ID` | Google Cloud project ID | `my-project-123456` |
| `GCP_LOCATION`, `GEMINI_LIVE_MODEL`, `GEMINI_VOICE` | Gemini Live region, model and optional voice | `us-central1`, `gemini-live-2.5-flash-native-audio` |
| `EMBEDDING_MODEL`, `EMBEDDING_LOCATION` | Must match the API's `VERTEX_EMBEDDING_MODEL` | `gemini-embedding-001`, `asia-south1` |
| `SEARCH_TIMEOUT_MS` | Give up on a library lookup after this long | `4000` |
| `MAX_CONCURRENT_SESSIONS`, `MAX_SESSION_MINUTES`, `IDLE_TIMEOUT_SECONDS` | Limits | `50`, `15`, `120` |
| `LOG_LEVEL` | `debug`, `info`, `warn` or `error` | `info` |

The relay's `README.md` documents the WebSocket message format and close codes for anyone working on the client.

## Aira web app (branch `aira-frontend`)

The web app is the only part users see. It needs two URLs, the API's and the relay's, and both are built into the JavaScript at build time.

## What it does

| Page | Purpose |
|---|---|
| Login and OTP verification ( `/login` , `/otp-verification` ) | Mobile number and OTP. A returning user gets a token; a new user goes on to registration. |
| Registration ( `/registration` ) | Name, email, date of birth, gender, address, state, city, pincode, income and an optional ID proof |
| Home ( `/` ) | The voice and text chat with Aira: the introductory message, push-to-talk microphone, typed messages and live transcripts |
| Session history ( `/session-history` ) | Past conversations, with title, summary and transcript, and a rating for each |
| Profile ( `/profile` ) | View and edit the user's details |

The interface is translated into English, Hindi, Marathi and Bengali ( `src/locales/` ). Microphone capture uses an audio worklet ( `public/worklets/pcm-capture.js` ) that records 16 kHz audio, and `src/hooks/useVoiceSession.ts` handles the relay connection and playback.

## Setup

1. Check out the branch and install the packages.

```
git clone -b aira-frontend --single-branch
https://github.com/AditiHazraGanju/AIRA.git aira-frontend
cd aira-frontend
npm ci
```

2. Create the environment file and fill in every `YOUR_*` value.

```
cp .env.example .env.local
```

3. Start the development server, then open `http://localhost:3000` .

```
npm run dev
```

4. For production, build after setting the variables, then serve the build.

```
npm run build
npm start
```

The microphone only works in a secure context: `https://`, or `http://localhost` during development.

### Environment variables

| Variable | What to put | Example |
|---|---|---|
| `NEXT_PUBLIC_API_BASE_URL` | API base URL including `/api/v1`, no trailing slash | `http://localhost:8000/api/v1` |
| `NEXT_PUBLIC_RELAY_WS_URL` | Relay WebSocket URL ending in `/ws`. Use `wss://` in production, because browsers block `ws://` on https pages. | `ws://localhost:8080/ws` |
| `NEXT_PUBLIC_TCMLOGGER_*` | Optional remote logging. Off unless both the base URL and the token are set. | |

Every `NEXT_PUBLIC_*` value is visible in the browser. Never put a server-side secret in one, and rebuild after changing them.

## Running everything together

Start the projects in this order: database, API, relay, admin panel, web app. Each one needs the one before it.

1. PostgreSQL with the `aira_db` schema loaded.
2. API on port 8000: `php artisan serve --port=8000`
3. Relay on port 8080: `npm start`
4. Admin panel on port 8001: `php artisan serve --port=8001`
5. Aira web app on port 3000: `npm run dev`

**Values that must match across projects**

| Value | Set in | Must equal | If it doesn't |
|---|---|---|---|
| `JWT_SECRET` | API | Relay `JWT_SECRET` | Every user is disconnected right after login (close code 4003) |
| `ADMIN_API_KEY` | API | Admin `ADMIN_API_KEY` | Every admin page shows empty tables (API returns 401) |
| `VERTEX_EMBEDDING_MODEL` | API | Relay `EMBEDDING_MODEL` | Content Library search fails (API returns 409) |
| API URL | API `APP_URL` | Admin `AIRA_API_URL` (no `/api/v1`), relay `LARAVEL_API_URL` and web app `NEXT_PUBLIC_API_BASE_URL` (both with `/api/v1`) | Calls fail to connect |
| Relay URL | Relay `PORT` and host | Web app `NEXT_PUBLIC_RELAY_WS_URL` | Chat shows "Couldn't connect to Aira" |
| Web app URL | Where Aira is served | Relay `ALLOWED_ORIGINS` | The relay refuses the WebSocket |

### End-to-end check

- [ ] In the admin panel, add a short text item to the Content Library and wait until it shows Ready. Indexing needs the Google Cloud key.
- [ ] Add a guardrail and an introductory message in English.
- [ ] Open the web app and log in with a new mobile number. With the API's `APP_ENV=local`, the OTP is `102025`.
- [ ] Complete registration and land on the home page. The introductory message plays.
- [ ] Ask a question about the content you added, by voice and by typing. Aira answers from it.
- [ ] Ask about the guardrail topic. Aira refuses.

- [ ] End the conversation and open Session history. The conversation appears with a title and summary.
- [ ] In the admin panel, Conversation Reports and SH Users show the new conversation and user.

## Security and production checklist

The two settings that can expose user data if you get them wrong are the admin panel's network access and the API's `APP_ENV`; check both on every deployment.

- [ ] **The admin panel is not public.** It has no login: anyone who can open it can change what Aira says and read every conversation. Put it on a private network, behind a VPN, or behind your web server's authentication (HTTP basic auth or an identity-aware proxy).
- [ ] **The API's `APP_ENV` is not `local`.** With `local`, every OTP is 102025 and anyone can log in as anyone.
- [ ] `APP_DEBUG=false` in the API and the admin panel.
- [ ] The API's `APP_KEY` stays the same for the life of the database, because it encrypts users' mobile numbers. Back it up with the database.
- [ ] `JWT_SECRET` and `ADMIN_API_KEY` are long random values, stored only in the `.env` files of the projects that need them. Changing `JWT_SECRET` logs every user out.
- [ ] The service-account JSON key sits outside every `public/` folder and outside git, readable only by the app's user.
- [ ] HTTPS everywhere: the web app, the API and the relay (`wss://`). The microphone needs HTTPS.
- [ ] The relay's `ALLOWED_ORIGINS` lists only the web app's URL(s).
- [ ] `.env` and `.env.local` files are never committed. Every branch's `.gitignore` already excludes them.
- [ ] Run `composer audit` and `npm audit` in each project and update the affected packages; the current lock files carry known advisories.

For production hosting, the API and admin panel need PHP-FPM behind Nginx or Apache, with the document root set to `public/`. The relay needs a long-running Node.js process, such as a VPS, a container or Cloud Run; shared or static hosting can't hold WebSockets. The web app can run with `npm start` behind a reverse proxy, or on any Next.js host.

# Troubleshooting

Most setup problems come from a value that differs between two projects; check the matching table in Running everything together first.

| Symptom | Likely cause | Fix |
|---|---|---|
| Admin pages load but every table is empty | `ADMIN_API_KEY` differs between the admin panel and the API, or `AIRA_API_URL` is wrong | Make both keys identical and check the URL has no `/api/v1`. The admin's `storage/logs/laravel.log` shows the API's 401. |
| "Your session has expired" right after login (relay closes with 4003) | The relay's `JWT_SECRET` differs from the API's, or the two servers' clocks are more than 60 s apart | Copy the API's `JWT_SECRET` into the relay and restart it. Compare the relay's startup `jwtSecretFingerprint` with the API's: `php artisan tinker --execute="echo substr(hash('sha256', config('jwt.secret')), 0, 12);"`. Turn on NTP time sync. |
| "Couldn't connect to Aira" in the chat | Wrong `NEXT_PUBLIC_RELAY_WS_URL`, relay not running, origin not in `ALLOWED_ORIGINS`, or `ws://` used on an https page | Open the relay's `/healthz`, fix the URL or origin list and rebuild the web app |
| Microphone doesn't start | Page not served over HTTPS | Use `https://`, or `http://localhost` in development |
| Content stays "Failed" in the Content Library | Google Cloud key missing or lacking permissions, or the bucket is wrong | Check `GOOGLE_CLOUD_KEY_FILE`, `GOOGLE_CLOUD_PROJECT_ID` and `GOOGLE_CLOUD_STORAGE_BUCKET`, then run `php artisan healthtalk:index-content` |
| Aira says she has no information about indexed content | Embedding model differs between the relay and the API (API returns 409), or search timed out | Make `EMBEDDING_MODEL` and `VERTEX_EMBEDDING_MODEL` the same, then re-index with `--all` if the model changed |

| Symptom | Likely cause | Fix |
| --- | --- | --- |
| Registration fails with "Please verify your mobile number with an OTP first" | More than 30 minutes passed since the OTP was verified | Log in again to get a new OTP |
| "No valid OTP found or OTP has expired" | The OTP is older than 5 minutes, or 5 wrong codes were entered | Request a new OTP |
| Returning users can't log in after a move to a new server | The API's `APP_KEY` changed, so stored mobile numbers no longer decrypt | Restore the original `APP_KEY` |
| `psql` fails on `CREATE EXTENSION vector` | pgvector isn't installed, or the user can't create extensions | Install pgvector on the server and run the schema as the database owner or a superuser |
| Admin pages have no styling | `npm run build` was not run in the admin project | Run `npm install && npm run build` |
| Changes to `NEXT_PUBLIC_*` have no effect | They are fixed at build time | Rebuild the web app |